Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3734477.3734710

RESEARCH-ARTICLE

Run-time Attestation and Auditing: The Verifier's Perspective

ADAM ILYAS CAULFIELD, University of Waterloo, Waterloo, ON, Canada

NORRATHEP RATTANAVIPANON, Prince of Songkla University, Hatyai, Songkhla, Thailand

IVAN DE OLIVEIRA NUNES, University of Zurich, Zurich, ZH, Switzerland

Open Access Support provided by:

Prince of Songkla University

University of Zurich

University of Waterloo

PDF Download
3734477.3734710.pdf
03 March 2026
Total Citations: 0
Total Downloads: 142

Published: 30 June 2025

Citation in BibTeX format

WiSec 2025: 18th ACM Conference on Security and Privacy in Wireless and Mobile Networks
June 30 - July 3, 2025
VA, Arlington, USA

Conference Sponsors:
SIGSAC

https://doi.org/10.1145/3734477.3734710
ISBN: 9798400715303

Run-time Attestation and Auditing: The Verifier's Perspective

Adam Ilyas Caulfield
University of Waterloo
Waterloo, Canada
acaulfield@uwaterloo.ca

Norrathep Rattanavipanon
Prince of Songkla University
Phuket, Thailand
norrathep.r@phuket.psu.ac.th

Ivan De Oliveira Nunes
University of Zurich
Zurich, Switzerland
ivan.deoliveiranunes@uzh.ch

Abstract

In run-time attestation schemes, including Control Flow Attestation (*CFA*) and Data Flow Attestation (*DFA*), a remote Verifier ($\mathcal{V}$rf) requests a potentially compromised Prover device ($\mathcal{P}$rv) to generate evidence of its execution control flow path (in *CFA*) and optionally execution data inputs (in *DFA*). Recent advances in this space also guarantee that $\mathcal{V}$rf eventually receives run-time evidence from $\mathcal{P}$rv, even when $\mathcal{P}$rv is fully compromised. Reliable delivery, in theory, enables *run-time auditing* in addition to attestation, allowing $\mathcal{V}$rf to examine run-time compromise traces to pinpoint/remediate attack root causes. However, $\mathcal{V}$rf's perspective in this security service remains unexplored, with most prior work focusing on the secure generation of authentic run-time evidence on $\mathcal{P}$rv.

In this work, we argue that run-time attestation/auditing is only effective if $\mathcal{V}$rf can analyze the received evidence. From this premise, we characterize different types of evidence produced by run-time attestation/auditing architectures based on $\mathcal{V}$rf's ability to use them for vulnerability detection/remediation. As a case study, we propose *SABRE*: a Security Analysis and Binary Repair Engine that enables $\mathcal{V}$rf to use run-time evidence to detect control flow attacks, to pinpoint specific instructions that corrupted control data, and to automatically generate binary patches to buffer overflow and use-after-free vulnerabilities without source code knowledge.

CCS Concepts

• **Security and privacy → Systems security**.

Keywords

Embedded Systems, Control Flow Attestation, Run-time Auditing

ACM Reference Format:
Adam Ilyas Caulfield, Norrathep Rattanavipanon, and Ivan De Oliveira Nunes. 2025. Run-time Attestation and Auditing: The Verifier's Perspective. In *18th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec 2025), June 30-July 3, 2025, Arlington, VA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3734477.3734710

1 Introduction

Embedded devices exist at the edge of larger systems, often serving as remotely operated sensors and on-demand actuators. Due to low energy and cost requirements, they are commonly implemented using low-end micro-controller units (MCUs) that lack advanced architectural security features (e.g., virtual memory or memory management units). Consequently, MCUs are more vulnerable and frequently targeted by cyber attacks [31, 40].

Given their low budget for preventative security features, MCUs need inexpensive means to prove their software integrity to back-ends that rely on their services (e.g., a device owner or control center). Towards this goal, Remote Attestation (*RA*) [16] has been proposed to give a resource-rich back-end – called a Verifier ($\mathcal{V}$rf) – the ability to remotely assess the software state of a resource-constrained Prover MCU ($\mathcal{P}$rv). In *RA*, $\mathcal{V}$rf challenges $\mathcal{P}$rv to return a cryptographic proof of $\mathcal{P}$rv's currently installed software binary. Only after receiving an authentic proof will $\mathcal{V}$rf trust that $\mathcal{P}$rv is installed with the correct software.

Although *RA* can effectively detect illegal code modifications, it cannot detect run-time attacks that do not modify code [1]. For example, an adversary ($\mathcal{A}$dv) could exploit a memory vulnerability (e.g., a buffer overflow [17]) to corrupt *control* data, including return addresses, indirect jump targets, and function pointers. Consequently, $\mathcal{A}$dv can chain out-of-order instruction sub-sequences to execute (often Turing-complete) malicious behavior [5, 61]. Since these attacks do not modify code, they are oblivious to *RA*.

To address this limitation, *RA* can be extended into Control Flow Attestation (*CFA*). In *CFA*, $\mathcal{V}$rf obtains cryptographic proof of both the installed software image and the exact control flow path followed in the most recent execution of the software [2]. This evidence and the respective proof are generated/authenticated by a root of trust (RoT) within $\mathcal{P}$rv that securely records control flow transfer destinations. Therefore, *CFA* evidence can, in principle, allow $\mathcal{V}$rf to detect both code modification and control-flow hijacking attacks. Recent architectures augment *CFA* to obtain *run-time auditing* [10, 12] by guaranteeing the delivery of *CFA* evidence to $\mathcal{V}$rf even when $\mathcal{P}$rv's software is compromised. See Sec. 2 for details on *CFA* and run-time auditing RoTs.

We observe that prior work in this area [2] has focused on secure design/implementation of $\mathcal{P}$rv RoTs, often overlooking $\mathcal{V}$rf's role. To our knowledge, no systematic analysis has explored $\mathcal{V}$rf's ability to interpret *CFA* evidence beyond basic path validation (i.e., determining whether the reported control-flow path is valid). However, in practice, after deeming a path invalid (thus $\mathcal{P}$rv compromise), $\mathcal{V}$rf would need to use *CFA* evidence to (1) identify the root cause of this compromise and (2) implement patches to mitigate it. Neither of these has been thoroughly investigated in the literature. The challenge increases when $\mathcal{P}$rv's software contains proprietary dependencies/third-party libraries with unavailable source code: a common occurrence in embedded system software chains.

Given the prevalence of memory-unsafe languages in embedded software and their various associated exploits [15], we argue that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WiSec 2025, Arlington, VA, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1530-3/2025/06
https://doi.org/10.1145/3734477.3734710

automated methods to identify unknown memory safety vulnerabilities from attested run-time evidence can result in timely detection of otherwise oblivious attacks and elimination of their root causes. While techniques have been proposed for root cause vulnerability analysis in high-end devices (e.g., based on Intel PT traces [30]), none of them can be used to analyze execution traces produced by *CFA*. In particular, these techniques [37, 80, 81, 83] require initial memory snapshots for a remote server to perform a root cause analysis. This requirement is unlikely to be satisfied in the *CFA* settings for two reasons. First, this would require $\mathcal{P}$rv to transmit potentially large memory snapshots during *CFA*, conflicting with the resource-constrained nature of MCUs [2]. Second, this snapshot must be stored by $\mathcal{P}$rv during the attested execution. This, in turn, unnecessarily reduces memory available to regular tasks on $\mathcal{P}$rv. We expand on these points in Sec. 6. Due to these conflicts, existing root cause analysis frameworks are not directly compatible with *CFA*. Our work bridges these gaps with the following contributions:

(1) We analyze and classify the types of run-time evidence generated by existing *CFA* RoTs. Their trade-offs are examined in detail with particular focus on their ability to support $\mathcal{V}$rf's remote vulnerability detection and remediation.
(2) As a case study, we present *SABRE*: a Security Analysis and Binary Repair Engine based on *CFA* evidence. *SABRE* combines binary analysis with *CFA* run-time evidence to automatically locate and patch buffer overflow and use-after-free vulnerabilities without requiring source-code knowledge.
(3) Prior root cause analysis assumes $\mathcal{V}$rf has the entire execution context, including an initial memory snapshot. As discussed earlier, this assumption does not hold for *CFA*. To address this, *SABRE* takes a different approach: upon detecting corrupted control data (e.g., return addresses or indirect call targets), *SABRE* performs an additional backward definitions analysis to reconstruct a *symbolic* memory state leading to the corruption. Since the recovered state is not fully concrete, existing root cause analysis techniques cannot be directly applied. *SABRE* overcomes this by extending prior work's forward symbolic data flow analysis to handle symbolic information. Based on this analysis and the detected vulnerability, *SABRE* automatically generates binary patches and validates their effectiveness.
(4) We develop an open-source prototype of *SABRE*, supporting TI MSP430 and ARM Cortex-M MCU binaries [11] and run-time evidence generated by existing open-source run-time attestation RoTs (based on Trusted Execution Environments (TEEs) [10] and custom hardware [12]).

2 Background

2.1 Remote Attestation

RA is a challenge-response protocol between two parties: a remote $\mathcal{V}$rf and a potentially compromised $\mathcal{P}$rv. It allows $\mathcal{V}$rf to remotely assess $\mathcal{P}$rv trustworthiness by measuring the content of $\mathcal{P}$rv program memory. As depicted in Fig. 1, a typical *RA* interaction involves the following steps:

(1) $\mathcal{V}$rf requests attestation from $\mathcal{P}$rv by sending a cryptographic challenge *Chal*.
(2) Upon receiving *Chal*, $\mathcal{P}$rv computes a measurement over its program memory and *Chal* to produce report *H*.

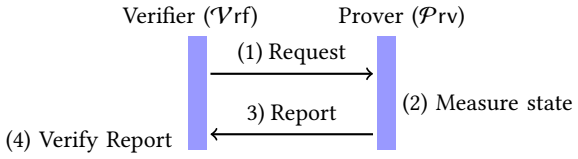


Figure 1: *RA* interaction

(3) $\mathcal{P}$rv sends the report *H* back to $\mathcal{V}$rf.
(4) Upon receiving *H*, $\mathcal{V}$rf checks *H* against the expected value to determine if $\mathcal{P}$rv has been compromised.

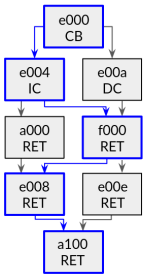
Early "software-based" attestation [32, 63–65] methods performed step (2) without relying on secrets and instead incorporated measurable architectural side-effects (e.g., execution/response time) along with the measurement of program memory. To make attestation more suitable to remote wireless connections with unpredictable network latency, subsequent methods required $\mathcal{P}$rv to respond with an authenticated integrity token — e.g., a message authentication code (MAC) or a digital signature – computed over program memory using a secret key (either a pre-shared symmetric key or the private key for a public key $\mathcal{V}$rf knows). Since threat models of *RA* (and of *CFA* consequently) assume that $\mathcal{P}$rv is susceptible to full software compromise, the secret key used in this operation must be securely stored by $\mathcal{P}$rv's RoT. Secure storage for the *RA* secret key implies some level of hardware support for the RoT implementation, ensuring that the key is unmodifiable and inaccessible to any untrusted software running on $\mathcal{P}$rv. Therefore, "hardware-based" approaches incorporate dedicated hardware RoT for storing secret keys and computing the measurement [36, 49, 56, 62, 72]. Since such hardware is too costly for low-end MCUs, recent approaches have proposed "hybrid" RoTs [6, 24, 50], which use hardware for storing secret keys and trusted software for performing the authenticated measurement.

2.2 Code Reuse and Control Flow Attacks

Control flow attacks are made possible due to an exposed memory vulnerability, such as common and pervasive instances like buffer overflow and use-after-free vulnerabilities [39]. Buffer overflows are common in embedded systems software since it is often written in unsafe memory languages like C/C++. For example, recent CVE's [43, 44, 46] have disclosed buffer overflows exploited for arbitrary code execution in IoT devices [60] and embedded real-time operating systems [84]. Use-after-free is another class of vulnerabilities that can lead to full software compromise. As it is not uncommon for embedded software and RTOSes to use dynamic (i.e., heap) allocation [19, 38, 58], they can also be vulnerable to use-after-free attacks, which is evidenced by recently disclosed CVE's [41, 45]. Since these attacks do not require code modifications, they would remain undetected by classic *RA* mechanisms.

2.3 Run-time Attestation and Auditing

Run-time Attestation and Auditing are a set of security services that aim to generate and guarantee the delivery of accurate and authentic evidence of a $\mathcal{P}$rv's run-time behavior to a remote $\mathcal{V}$rf, thus reflecting to $\mathcal{V}$rf whether any control flow attack has occurred. One type of run-time attestation is Control Flow Attestation (*CFA*) [1, 3, 21, 29, 48, 53, 67, 71, 79, 82, 88], which extends *RA* to



(a) Hash-based	(b) Verbatim Trace	(c) Hybrid
$H_0 = H(e004 \oplus 0)$	T = [e004]	$T_F = [0]$ $H_0 = 0$
$H_1 = H(f000 \oplus H_0)$	T = [e004, f000]	$T_F = [0,f000]$ $H_0 = 0$
$H_2 = H(e008 \oplus H_1)$	T = [e004, f000, e008]	$T_F = [0,f000]$ $H_1 = H(e008 \oplus H_0)$
$H_{CF} = H(a100 \oplus H_2)$	T = [e004, f000, e008, a100]	$T_F = [0,f000]$ $H_R = H(a100 \oplus H_1)$
E1 = H_{CF}	**E2 = T**	**E3 = $[T_F, H_R]$**

Figure 2: A conditional branch (CB), indirect/direct calls (IC/DC), and return (RET) in (a) hash-based, (b) verbatim, and (c) hybrid evidence.

detect run-time attacks in addition to illegal binary modifications. Upon receiving a request to execute some software function along with a *RA* challenge from $\mathcal{V}$rf, $\mathcal{P}$rv executes the requested function, and a RoT in $\mathcal{P}$rv records a trace of this execution into protected memory. In the most fine-grained case, the trace is a control flow log (CF_{Log}) of all control flow transfers that have occurred during the requested execution. After producing CF_{Log}, $\mathcal{P}$rv performs the authenticated *RA* integrity measurement over the CF_{Log}, the received challenge, and its own program memory to produce a timely and authenticated report. This report attests to the static state of $\mathcal{P}$rv's program binary and the run-time state within this timely execution. When applicable, data inputs – in Data Flow Attestation (*DFA*) [21, 52, 67] – or produced outputs [8, 51] can also be included in this proof to bind obtained results to the proper execution of respective software functions on expected inputs.

Existing *CFA* techniques use either (1) binary instrumentation along with TEE support; or (2) custom hardware modifications to generate CF_{Log} by detecting and saving each branch destination to a dedicated and protected memory region. For techniques that use binary instrumentation, a pre-processing phase modifies the binary so that all branch instructions (e.g., `jumps`, `returns`, `calls`, etc.) are prepended with additional calls to a TEE-protected trusted code. Once called, the trusted code appends CF_{Log} to update the trace with the current branch destination. In hardware-based techniques, custom hardware interfaces with the MCU core to detect branches and records their destinations to a reserved memory region. Recent work [10, 12] has also created mechanisms to implement reliable communication with $\mathcal{V}$rf as a part of the *CFA* RoT, enabling *run-time auditing* by ensuring evidence delivery even in the presence of malware that infects $\mathcal{P}$rv and refuses to deliver evidence about its actions/presence.

3 *CFA* Evidence and its Utility

3.1 Evidence Types

The various realizations of *CFA* RoTs (see Sec. 2) create and manage CF_{Log} differently, presenting trade-offs between the storage/-communication cost and ease of verification/analysis by $\mathcal{V}$rf. We classify existing methods in three types: hash-based evidence (**E1**), verbatim evidence (**E2**), and hybrid evidence (**E3**). Compared to Ammar et al. [2], we further offer a more detailed analysis of each type's suitability for advanced analysis.

Hash-based Evidence (E1). As control flow transfers occur, branch destination addresses are accumulated into a single hash chain. This is performed by first accumulating the current destination address with the previous hash and then hashing the accumulated result [1, 20, 82], as exemplified in Fig. 2.a. After the first branch occurs, its destination `0xe004` is accumulated into the hash. Since this is the first branch destination, it is accumulated with an initial constant to obtain H_0. Upon the second branch instruction, its destination address `0xf000` is accumulated with H_0 to obtain H_1. This process continues until the end of the requested execution generating a unique value that represents the path. Although simple control flows produce a single hash in **E1**, programs with more complex control flows (including those with nested loops and branching) might require a set of hash digests [1, 82]. This approach yields minimal storage/transmission overheads (i.e., the fixed size of a hash digest). However, $\mathcal{V}$rf must perform its analysis solely based on the final hash digest and the executable binary.

Verbatim Evidence (E2). On the other side of the spectrum, more recent *CFA* techniques advocate for storing and transmitting CF_{Log} in its entirety [10, 12, 21, 53, 71, 88]. As depicted in Fig. 2.b, the full sequence of destination addresses is maintained in a buffer. Therefore, this approach requires further (lossless) compression or is limited to small and self-contained operations to avoid filling the dedicated storage too quickly. A common strategy is to log simple loops (those without internal branches) with their address once and a count/condition pertaining to the number of iterations performed [10, 12, 53, 88]. Another reduction is obtained by recording statically defined branch destinations as a single bit [21]. Since static destinations can be determined from the binary alone, branch instructions with statically defined destinations can be recorded as a single bit (1/0) to determine if the branch was taken or not. Branches with dynamically defined destinations (e.g., returns and indirect calls) are still recorded with their full addresses. If CF_{Log} remains larger than available storage despite these optimizations, it can be split into a set of multiple fixed-size slices and streamed to $\mathcal{V}$rf, as suggested in [10, 12, 71]. Despite the aforementioned challenges, **E2** provides $\mathcal{V}$rf with full control flow path information for analysis.

Hybrid Evidence (E3). Some techniques implement a combination of **E1** and **E2**. OAT [67] and ARI [73] record forward edges *verbatim* and accumulate backward edges (returns) into a single hash chain. As shown in Fig. 2.c, the RoT in $\mathcal{P}$rv maintains a trace of forward edges (T_F) and a hash chain of returns (H_R). Since `0xf000` is the destination of an indirect call, its full address must be recorded whereas the first destination can be represented by a single bit '0' encoding the 'not-taken' conditional branch address. After the attested execution ends, T_F and H_R are sent to $\mathcal{V}$rf.

3.2 Detection & Remediation via E1, E2 & E3

Consider an attack detection strategy in which $\mathcal{V}$rf constructs the CFG of the executable binary, i.e., as suggested in [1, 12, 20, 71, 82, 88]. An initial set of *invalid paths* is defined as forward edges that do not exist in the statically constructed CFG. As valid indirect targets (e.g., return addresses or indirect calls) are over-approximated using the static CFG [7], $\mathcal{V}$rf may also use CF_{Log} along with the program's binary to emulate the execution of the program locally.

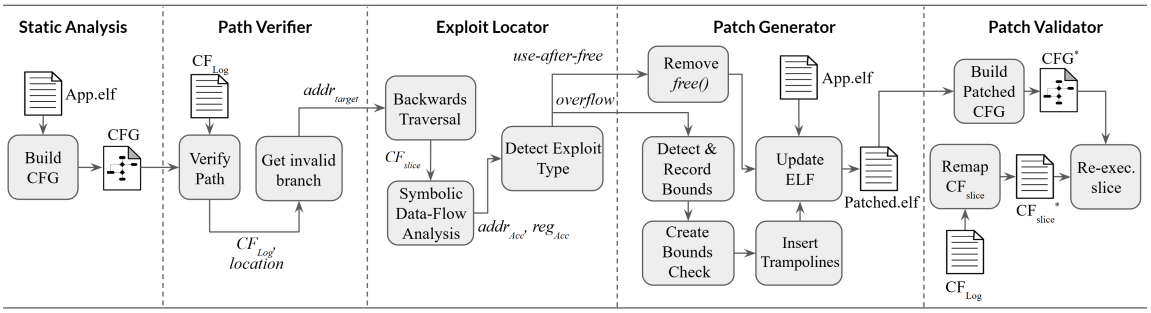


Figure 3: *SABRE* sub-modules and internal workflow

In this process, a Shadow Stack (SS) can be emulated locally to verify backward edges (return destinations) and data flow (DF) analysis can be employed to determine a set of valid call sites for each indirect call, updating the CFG accordingly. We observe feasibility of this verification strategy varies greatly depending on evidence types **E1**, **E2**, and **E3**.

For **E1**, with the only evidence available being a hash digest, $\mathcal{V}$rf must infer which path has resulted in the digest. To that end, it must generate each possible legal path based on the CFG and use an emulated SS to accumulate the path's hash during each step of the CFG traversal. However, the complexity of this task grows exponentially with the number of branches, leading to the intractable path explosion problem [4, 59]. Considering anomalous/illegal paths, the analysis is even more challenging. $\mathcal{V}$rf would need to map the hash digest to the exact illegal path. Illegal paths do not exist in the CFG and thus must then be enumerated directly, making it effectively impossible to learn about the exploit behavior from this type of evidence (this precludes any analysis of exploit root causes, as discussed below).

Evidence types **E2** and **E3** allow $\mathcal{V}$rf to use reported traces to traverse the CFG, checking the reported destination against valid destinations. When there is a mismatch between the reported destination and the set of valid destinations for that branching instruction, $\mathcal{V}$rf determines that $\mathcal{P}$rv executed an invalid path. With both **E2** and **E3**, $\mathcal{V}$rf is able to emulate an SS of return addresses. For verifying the integrity of returns in **E2**, $\mathcal{V}$rf checks each return destination individually as they are reached in the path traversal. The expected return address is popped from the SS and compared to the return address reported in CF_{Log}, and an invalid path is detected when there is a mismatch. For **E3**, however, $\mathcal{V}$rf must recompute all returns reached into the final hash chain digest and compare it to the one reported in CF_{Log}. If there is a mismatch of hashes, an invalid path is detected due to one of the return addresses. We note that verification based on **E3** is only able to determine that some return address was corrupted. Since actual return addresses are not logged *verbatim*, the path explosion problem still applies when attempting to determine which return in the chain was corrupted, preventing accurate root cause analysis by $\mathcal{V}$rf.

The discussion above highlights that $\mathcal{V}$rf's ability to learn an attack's behavior and its root cause varies. Put simply, **E1** only allows $\mathcal{V}$rf to learn if the reported path is invalid. However, it is impossible to learn what the invalid control flow path was or how it was exploited.

With **E2**, an invalid path is determined when a forward-edge target does not match possible successors in the CFG or when a return address in **E2** does not match the value popped from the SS. In either case, $\mathcal{V}$rf can determine the exact indirect call or return exploited since all of these destinations are included in CF_{Log}.

In **E3**, since $\mathcal{V}$rf only receives a hash chain of all return addresses, it cannot determine which return was corrupted or its destination address after the corruption. As a consequence, $\mathcal{V}$rf cannot make sense of subsequent forward edges in CF_{Log} because it cannot verify that $\mathcal{P}$rv has followed the expected control flow transfers (as a previously exploited return could have gone anywhere). At this point, continuing to traverse the CFG yields no additional information. Thus, $\mathcal{V}$rf can not learn anything more about the execution, resulting in limited analysis capability beyond detecting that some control deviation occurred. In sum, similar to **E1**, **E3** results in $\mathcal{V}$rf being unable to examine exploit root causes.

It follows from the discussion above that **E2** is the most adequate to support auditing and exploit root cause analysis by $\mathcal{V}$rf based on CF_{Log}. Based on this conclusion, Sec. 4 presents *SABRE* to showcase how $\mathcal{V}$rf analysis can be performed based on a CF_{Log} when it includes a *verbatim* sequence of control flow transfers.

4 *SABRE*

SABRE showcases CF_{Log}-based root cause analysis and remediation with *verbatim* control flow traces (**E2**) and without the source-code of the attested program. Fig. 3 shows *SABRE* workflow consisting of five internal modules: Static Analysis, Path Verifier, Exploit Locator, Patch Generator, and Patch Validator. *SABRE* takes as input an application binary (*App.elf*) and a verbatim CF_{Log} (**E2** evidence type). The notation used in this work is summarized in Table 1.

4.1 Static Analysis and Path Verifier

The first two stages of *SABRE* are well studied from prior works [2]. The first module of *SABRE* performs static analysis of the application binary (*App.elf*) to produce the CFG used by subsequent modules. Each CFG node represents a set of contiguous (non-branching) instructions starting with the previous branch destination and ending with the next branch, as shown in Fig. 4. This module is executed once per *App.elf*. The remaining modules are executed for each received CF_{Log}.

SABRE verifies the contents of the verbatim CF_{Log}, the contents of which are assumed to have the format depicted in Fig. 4. *SABRE* interprets each CF_{Log} entry as either: (1) a branch destination (denoted $addr_{Ni}$ in Fig. 4) or (2) a loop counter (denoted $loop_{count}$

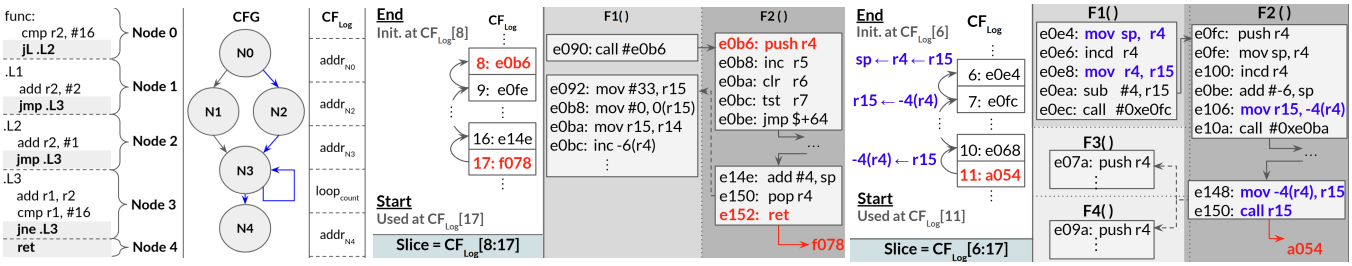


Figure 4: CFG and CF_{Log} **Figure 5: Backward Traversal for a `ret`** **Figure 6: Backward Traversal for an `call`**

Table 1: Notation Summary

Symbol	Description
CF_{Log}	Verbatim CFA evidence (type **E2**)
App.elf	The binary of the attested application
CFG	The Control Flow Graph of *App.elf*
SS	Shadow Stack of return addresses
$loop_{count}$	a special entry in CF_{Log} to optimize static loops (i.e., loops with no internal branches) [12, 53]; denotes total times the previous CF_{Log} entry repeated
$addr_{target}$	The control data (e.g., return address, indirect jump/call target) that was corrupted
CF_{Slice}	A CF_{Log} slice from $addr_{target}$'s initialization point its corrupted usage.
base	The symbol (*i.e., the base* address or register) storing $addr_{target}$
MemMap	address-value mapping for in-use memory addresses during Symbolic DF Analysis.
RegMap	register-value mapping for in-use registers during Symbolic DF Analysis.
FreeList	list of all freed pointers during Symbolic DF Analysis
X	special symbolic value assigned to *base* during Symbolic DF Analysis.
$addr_{Acc}$	the address of the memory instruction used to corrupt $addr_{target}$
reg_{Acc}	the register $addr_{Acc}$ dereferences to corrupt $addr_{target}$
reg_{Acc}^{Init}	the initial value of reg_{Acc} before it is modified to dereference/corrupt $addr_{target}$. In the context of buffer overflows, it holds the start address of the overflown buffer.
$addr_{lower}$	the instruction address that sets the lower bound of the buffer
$addr_{upper}$	the instruction address that sets the upper bound of the buffer

in Fig. 4), indicating the number of times that a CFG node in the previous entry has executed. After receiving CF_{Log}, Path Verifier uses this information to determine whether CF_{Log} constitutes a valid control flow path by traversing the CFG and SS, as described in Sec. 3.2. Upon detecting an invalid transfer, it proceeds to analyze the type of control flow instruction (i.e., `ret` or indirect `call`) whose destination address is being corrupted. This is done by first locating the node in the CFG that aligns with the last valid transfer in CF_{Log}. The corrupted instruction is assigned as the last instruction of this CFG node. Using *App.elf*, the corrupted instruction and target address ($addr_{target}$) are identified as either a return address or an indirect branch target. This module outputs the corrupted instruction and $addr_{target}$ to the next module of *SABRE*.

4.2 Exploit Locator

Phase 1. Backwards Traversal. Given the corrupted instruction from the Path Verifier, this phase attempts to find:

(1) CF_{Slice}: the section of CF_{Log} relevant to this particular exploit, including the CF_{Slice} entries between the time when memory storing $addr_{target}$ was assigned with the last pre-corruption value and the time that the corrupted branch executes causing the detected violation.
(2) *base*: the memory address storing $addr_{target}$ at the beginning of CF_{Slice}.

The procedure for determining CF_{Slice} and *base* varies based on the type of $addr_{target}$. This is shown in Fig. 5 and Fig. 6.

Corrupted returns. Return addresses are initialized when pushed onto the stack during function calls, and they remain unmodified until the respective returns. Consequently, any corruption to a return address must have occurred between the function call and the return instruction. Therefore as a first step, *App.elf* is inspected to determine the start address of the function containing the corrupted return. In the example from Fig. 5, Path Verifier detects that the 17th CF_{Log} entry is invalid, corresponding to the corrupted return instruction at address `0xe152` and $addr_{target}$ pointing to `0xf078`. This module then inspects the disassembled instructions of *App.elf* and identifies that function `F2` contains this corrupted instruction. Then, it saves the start address of `F2`, `0xe0b6`, as the target of the backward traversal. A backward traversal of CF_{Log} is performed to find the last call to `F2`, which happens to be at the 8th CF_{Log} entry. As a result, this phase returns CF_{Slice} consisting of entries 8 to 17, i.e., CF_{Log}`[8:17]`. In addition, since return addresses are always initialized on the stack, *base* can be found in the stack pointer register (sp) at the beginning of CF_{Slice}. Hence, this phase also outputs *base* to be sp.

Corrupted forward edges. Indirect forward edge targets (indirect jumps/calls) are stored in a register operand at the time of their use. To identify $addr_{target}$'s initialization point in this case, CF_{Log} is used for a backward traversal of CFG while repeatedly following its assignments until reaching its initialization. For example, in Fig. 6, Path Verifier determines that, at instruction address `0xe150`, $addr_{target}$ is stored in `r15`. Thus, this phase traverses instructions backward from address `0xe150` until it finds the latest assignment of `r15` at address `0xe148`. As `r15` is assigned by `-4(r4)` at this address, Exploit Locator switches to tracking this new memory instead. This process continues until it encounters the instruction that initializes $addr_{target}$, which then sets the beginning of CF_{Slice} to the first CF_{Log} entry that includes this instruction. This phase finally assigns *base* according to the type of the source operand in this instruction:

(1) the stack pointer (i.e., where $addr_{target}$ is declared as a local variable), which is the case in Fig. 6. In this example, this phase stops once it finds a match of the source operand with sp at `0xe0e4`. Since this instruction reflects the 6th entry of CF_{Log} and the corrupted instruction happens in the 11th, it returns $CF_{Slice} = CF_{Log}$`[6:11]`. It outputs *base* = sp.
(2) a constant indicating a fixed memory address (i.e., when $addr_{target}$ is declared as a global variable). *base* is set to this fixed address.
(3) a return value from *malloc* (i.e, in the case where $addr_{target}$ is dynamically allocated). Backward traversal stops at the last *malloc* that defined it. Thus, CF_{Slice} only includes the *malloc* and *free* that potentially lead to the attack. In this case, *base* is set to the return register from the *malloc*.

Phase 2. Symbolic DF Analysis. Within CF_{Slice} determined by Phase 1, Exploit Locator must locate the instruction that corrupts $addr_{target}$, typically requiring emulation of CF_{Slice} execution. Without access to run-time data prior to CF_{Slice} execution, Exploit Locator emulates this execution using symbolic analysis. Unlike

Algorithm 1: Symbolic Data Flow Analysis.

```
Input  : CF_Slice: CF_Log slice containing vulnerability;
         App.elf : The application binary;
         CFG: Control flow graph of App.elf;
         base : symbol holding addr_target at CF_Slice start;
Output : addr_Acc : address of illegal memory instruction;
         MemMap : memory map containing address-value pairs used in CF_Slice;
         RegMap : register map containing register-value pairs used in CF_Slice;
         FreeList : list of freed pointers during evaluation of CF_Slice;

MemMap, RegMap, FreeList ← ∅, ∅, ∅
MemMap ← AddConsts(App.elf)
if base is a Register then
    RegMap ← RegMap ∪ {(base, X)}
else
    MemMap ← MemMap ∪ {(base, X)}
end
foreach entry ∈ CF_Slice do
    if entry is not loop_count then
        dest_addr := entry
        max_iters := 1
    else
        max_iters := entry
    end
    foreach i ∈ {1, 2, ..., max_iters} do
        foreach instr_addr ∈ CFG[dest_addr] do
            MemMap, RegMap, FreeList ←
              eval(instr_addr, MemMap, RegMap, FreeList)
            if ∃(a, b) ∈ MemMap s.t. a = X then
                addr_Acc := instr_addr
                return: (addr_Acc, MemMap, RegMap, FreeList)
            else
        end
    end
end
```

traditional symbolic execution [35], we do not use symbolic analysis to explore different paths. Instead, we use it to reconstruct all possible data flows during CF_{Slice} execution [80, 81] and monitor for the change in data flows that result in corruption of $addr_{target}$.

Algorithm 1 details this phase. The symbolic data flow analysis takes CF_{Slice}, *base*, *App.elf*, and CFG as inputs. As a first step, it initializes the execution state using *MemMap*, *RegMap*, and *FreeList*, where:

- *MemMap* represents the set of memory states affected by CF_{Slice} execution, mapping memory addresses to their values. It is initially set to empty, except for addresses storing read-only data that can be determined from *App.elf* (e.g., data in `.rodata` segment of *App.elf*).
- *RegMap* is a map of registers and their values for registers used in CF_{Slice} portion of *App.elf* execution. In the initial state, *Reg* is empty.
- *FreeList* is a list of all pointers freed during the CF_{Slice} portion of *App.elf* execution. *FreeList* is also empty in its initial state.

The concrete value of *base* is only available during run-time and cannot be determined from CF_{Slice}. Therefore, this phase initializes *base* with a special symbolic value (X) and updates the state (*MemMap* or *RegMap*) in which *base* is stored accordingly. For instance, in Fig. 5 and 6, as *base* is found in `sp` at the start of CF_{Slice}, *Reg*[`sp`] is set to X.

After initialization, this phase proceeds with symbolic execution of CF_{Slice} by iterating through each *entry* in CF_{Slice}. Assuming that the current *entry* corresponds to a branch destination, it locates the CFG node containing this destination and extracts all instructions from it. Each instruction is then evaluated based on the current program state: if the instruction depends on memory or registers not currently present in *MemMap* or *RegMap*, symbolic values (independent from X) are assigned to those memory/registers before executing it; otherwise, the instruction is simply executed symbolically. As a result of symbolic execution, the evaluation updates *MemMap* and *RegMap* to reflect the new execution state. Additionally, *FreeList* is updated whenever an evaluated instruction calls *free()*. When this is detected (by inspecting the instruction type and its destination address), the input parameter for the *free()* is first obtained from the previous instruction. Then, the input parameter and the `call` instruction's address are added to the *FreeList*.

Next, Exploit Locator uses the evaluation results to determine whether a memory write to *base* has occurred in the current evaluation. This is done by checking if the value of X in *MemMap* has changed. If so, it indicates that the current instruction introduces a value (i.e., writes) to memory at *base*, leading to $addr_{target}$'s corruption. Exploit Locator then outputs the address of this instruction as $addr_{Acc}$, along with *MemMap*, *RegMap*, and *FreeList* to the next phase.

When no write to X is detected, it continues with symbolic execution over the remaining CF_{Slice} entries. We note that this analysis may encounter a CF_{Slice} entry that is a $loop_{count}$ (as depicted in Fig. 4). In this case, it repeats symbolic evaluation on the same CFG node for $loop_{count}$ times.

Phase 3. Detecting Exploit Type. In its final phase, the Exploit Locator attempts to determine whether the control flow attack resulted from a use-after-free or a buffer overflow, using the output from the previous phase.

The *FreeList* can be inspected to determine whether a use-after-free has occurred. Since the previous phase stopped at the point where *base* was overwritten, the Exploit Locator checks the contents of *FreeList* to see if it contains *base*. If *base* is in *FreeList* and it was illegally modified after it was freed, it is likely that *base* was corrupted due to a use-after-free. In this case, Exploit Locator outputs the candidate attack root cause as a use-after-free.

The Exploit Locator assumes that an attack exploits a buffer overflow when *base* is not in *FreeList* and when the CFG node containing $addr_{acc}$ has been executed multiple times in CF_{Slice} (implying that this CFG node is part of a loop that performs repeated memory writes, eventually overflowing into $addr_{target}$). In this case, Exploit Locator outputs the attack root cause as a buffer overflow.

4.3 Patch Generator

SABRE patching strategy [22, 25, 78, 87] redirects the vulnerable instructions to a secure implementation. The exact steps depend on the detected root cause (either use-after-free or buffer overflow).

4.3.1 Patching Use-After-Free. Two root causes lead to a use-after-free: the *free* and the *use* along CF_{Slice}. The *use* of a freed pointer typically occurs when a new object is allocated and when the same freed pointer (i.e., $addr_{target}$ in our case) is indirectly redefined through this new object. Without source-code semantics (such as variables, structs, pointer types/names), it is challenging to determine whether altering the *use* of $addr_{target}$ will lead to unwanted changes to a program's functionality. Therefore, *SABRE* targets the unintended *free* along slice for patching.

SABRE patches the use-after-free by selectively removing the offending *free* from *App.elf* by replacing it with `nop` instructions.

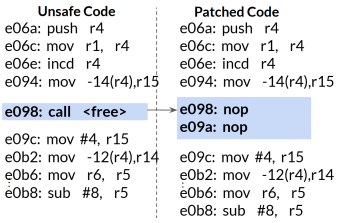


Figure 7: Example patch for use-after-free vulnerabilities

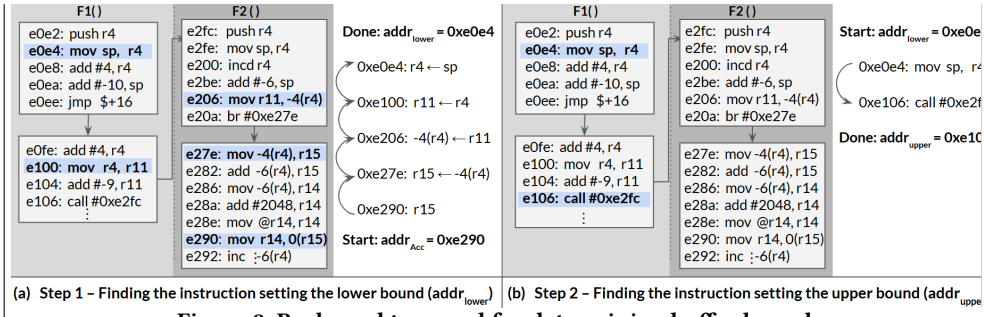


Figure 8: Backward traversal for determining buffer bounds.

An example of *SABRE*'s use-after-free patch is depicted in Fig. 7. In this example, call instruction at address `e098` was identified in the previous step as the call to *free()* in CF_{Slice} that frees $addr_{target}$. Call instructions (e.g., `call` in MSP430 and `bl` in ARM) take a full address as a parameter and thus span two memory addresses. Therefore, the `call` is replaced with two `nop` instructions.

4.3.2 Patching Buffer Overflows. Generating a patch for buffer overflow vulnerabilities includes the following tasks:

T1: Estimate the upper and lower bounds of the buffer that overflows into $addr_{target}$.

T2: Rewrite the instructions in the original code to safely record the bounds.

T3: Copy the function containing $addr_{acc}$ to an empty region of memory, wrapping $addr_{acc}$ in new instructions that perform a bounds check.

T4: Replace the original call to the function from **T3** with a trampoline to the patched version.

T1: Estimating Buffer Bounds. Buffer overflows are eliminated via bound checks before memory accesses according to the buffer size. From the previous phase, the Exploit Locator identified the offending instruction to be located at $addr_{Acc}$ and that it causes an overflow onto *base*. This suggests that a buffer exists next to *base* that must have been overwritten by a loop before the overflow occurs. It also implies that the offending instruction overflowing to *base* is the same as that used to write data to the buffer.

With these insights, backtracking to where the destination operand (storing the overflown address) is initialized should provide the start address of the overflown buffer. Using this address, valid buffer bounds can be estimated to insert bound checks to the binary accordingly. This phase aims at finding these bounds before a patch can be generated and applied in the next modules.

The example of Fig. 8 depicts the two steps required to obtain the buffer's bounds. The previous phase has detected that the instruction `mov r14, 0(r15)` triggers a write to *base*, setting $addr_{Acc}$ to `0xe290`. From this instruction, this phase knows that its destination register, `r15`, stores the write address pointing into the overflown buffer, and thus labels it as reg_{Acc}. To determine the start address of the overflown buffer, *SABRE* performs a backward traversal from `0xe290` until it finds the instruction that initializes its lower bound.

As depicted in Fig. 8, the buffer's definition is influenced by `-4(r4)` at `0xe27e`, which is in turn initialized by `r11` at `0xe206`. Tracking of definitions continues until a known pointer value is reached, as described in Sec. 4.2. In this example, definitions are tracked until `0xe0e4` since the stack pointer (a known pointer value) has been reached. Thus, Exploit Locator saves `0xe0e4` as $addr_{lower}$ (the address defining the lower bound of the exploited buffer) and saves the value of reg_{Acc} (`sp` in this example) as the lower bound of the overflown buffer at this instruction.

After this step, *SABRE* looks forward in CF_{Slice} to find an instruction that establishes an upper bound for reg_{Acc} used at $addr_{lower}$. For example, since reg_{Acc} is the stack pointer in Fig. 8, *SABRE* stops at the next `call` instruction at address `0xe106`, as the stack pointer value before this `call` will determine the upper bound of the current stack frame, providing an approximation of the buffer's upper bound. Therefore in this example, the address `0xe104` (the instruction before the `call`) is saved as the location that defines the upper bound ($addr_{upper}$). Consequently, the value of reg_{Acc} at $addr_{upper}$ is used as the upper bound. When $addr_{target}$ is defined within the same stack frame as reg_{Acc}, *base* is used as the buffer's upper bound. Similarly, when reg_{Acc} obtains its definition from another source (e.g., a fixed memory address or as an output from *malloc*), $addr_{upper}$ may not exist or cannot always be determined through this method. In this case, *SABRE* defaults to using *base* as the upper bound.

Although this strategy could overestimate (i.e., the estimated bounds may include other data objects and the buffer of interest), narrowing the bounds any further cannot be done without source-code level semantics. For example, the instruction at `0xe0ea` modifies the stack pointer by ten. However, it cannot be determined through the binary alone whether this is allocating space for a buffer of size ten or for all stack variables totaling ten memory addresses. Although this differentiation cannot be made through binary alone, *SABRE*'s approach guarantees overflows into $addr_{target}$ are blocked by restricting overflows within the buffer's stack frame.

T2: Recording the Bounds. After identifying $addr_{lower}$ and $addr_{upper}$, *SABRE* completes **T1** and has estimated the bounds of the buffer that overflows into $addr_{target}$. To complete **T2**, the values of reg_{Acc} must be saved at both locations. This is completed by placing a trampoline both at $addr_{lower}$ and $addr_{upper}$ to preserve their values in a dedicated register. Fig. 9 depicts the entire patch for buffer overflow vulnerabilities and shows how these instructions are added for recording the bounds.

In this example, the trampolines and new instructions to save the lower and upper bounds are depicted with the instructions highlighted in purple and orange, respectively. First, the original instructions at $addr_{lower}$ and $addr_{upper}$ are replaced with a trampoline into a new region in the binary. For the upper bound, placing the trampoline alters the alignment, thus the trampoline and a `nop` are inserted to replace two instructions. Once trampolined into the

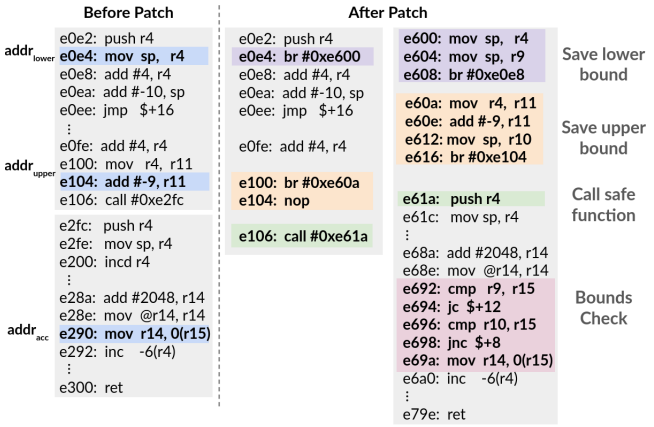


Figure 9: Example patch for buffer overflow vulnerabilities.

new region, the instructions that were replaced by the trampoline execute first. Then, reg_{Acc} (sp in this example) is recorded into reserved registers r9 and r10 to save the lower and upper bounds, respectively. To ensure the bounds cannot be altered, *SABRE* does a pass over *App.elf* binary to replace any other uses of these registers to ensure they are reserved. After the reserved registers have been updated with reg_{Acc} value, the new instructions trampoline back into the previous code.

T3: Inserting the Bounds Check Then, Patch Generator proceeds to patch the offending memory instruction by prepending it with instructions that check if the address being accessed is outside the valid boundary. If it is, the offending instruction is skipped. For the patch depicted in Fig. 9, the prepended instructions (highlighted in red) correspond to:

- `cmp r15, r9; jc $+12;` A comparison between the memory address being accessed (via r15) and the lower bound of the buffer (preserved in r9). If r15 is less than r9, it will skip the memory write by jumping to an offset of +12.
- `cmp r15, r10; jnc $+8;` A similar comparison except to the upper bound of the buffer (preserved in r10). If r15 exceeds the upper bound, the memory write will be skipped by jumping to an offset of +8.

The example in Fig. 9 uses instructions compiled for TI-MSP430, but the same patch is applied by *SABRE* to ARM-Cortex M33 binaries by using equivalent instructions. For example, instead of using jc and jnc, the patch targeting an ARM-Cortex M33 binary uses bcc and bhi, respectively.

To complete **T3**, the entire function that contains the exploited memory instruction ($addr_{acc}$) is rewritten to include this bounds check. The modified "safe" version of this function is placed into a new region in unused memory. We chose to make a safe copy and do not alter the original function in place to avoid altering all uses of this function (as the vulnerability is specific to this usage and does not necessarily apply to all calls in the code). As such, the final task **T4** ensures that the safe version of this function is called at the previous $addr_{target}$-corrupting call site $addr_{target}$.

T4: Calling the Safe Function. To realize **T4**, Patch Generator rewrites the original call instruction that leads to the function containing the buffer overflow. It changes this call instruction to instead call the patched version. This is depicted in Fig. 9 by the instructions highlighted in green.

The call instruction at 0xe106 previously called to the "unsafe" function at 0xe2fc. In the patched version, this call instruction now calls 0xe61a, the location of the patched version of the function. As depicted, this new function contains bounds-checking from **T3**.

4.4 Patch Validator

Before deploying the patched software on $\mathcal{P}$rv, it must be validated to ensure its effectiveness against the previously identified attack. To achieve this, this module first builds an updated CFG of the patched binary. Then, it remaps the destination addresses in CF_{Slice} to their respective addresses in the updated CFG. It also inserts new entries into CF_{Slice} as necessary to account for the patching behaviors (e.g., a trampoline to/from the patch). Patch Validator then performs symbolic DF analysis over the translated CF_{Slice}, similar to how it is done in Sec. 4.2. When the analysis reaches the end of CF_{Slice} without $addr_{target}$ being corrupted, Patch Validator considers this patch effective. When the corruption still occurs, the attack was likely caused by another vulnerability. In this case, *SABRE* generates a report indicating to $\mathcal{V}$rf that further manual analysis is required.

5 Implementation & Evaluation

SABRE was implemented as a combination of Bash and Python and evaluated on an Ubuntu 18.04 machine with an Intel Core i7-4790 CPU at 3.60GHz and 32GB RAM. BEEBs benchmark applications for embedded platforms [54] were used for the evaluation. They were compiled for MSP430 and ARM Cortex-M33 and executed atop unmodified *CFA* architectures, ACFA [12] and TRACES [10], to generate a dataset of executables and corresponding CF_{Log}-s. The MSP430 prototype was configured with 8 KB of application program memory, and the ARM Cortex-M33 prototype was configured with 256 KB of application program memory. We believe that *SABRE* concepts generalize to several CFA architectures and use ACFA and TRACES as case studies due to their open-source availability.

To obtain CFGs from executables, *SABRE* first runs *objdump* over the executables to decode the assembly instructions. Then, it builds the CFG from this disassembled file. *SABRE* uses SymPy [70] for evaluating symbolic expressions and Keystone assembler [33] for generating ARM Cortex-M instructions. Since no MSP430 assembler exists in Python packages, we build upon and modify MSProbe [68] for MSP430 instructions. Finally, *SABRE* uses the Python library ELFtools [28] to generate the final patched executable. *SABRE* proof of concept prototype is publicly available at [11].

5.1 Run-times of *SABRE* Sub-Modules

We evaluate the run-time of each *SABRE* sub-module after a malicious path was detected. Given the lack of reproducible exploits and incomplete CVEs for MCUs software (e.g., missing source codes, broken links, etc.) [69], we craft control flow attacks that exploit buffer overflow and use-after-free vulnerabilities, following a similar methodology to recent related work [14]. We then insert them into the example benchmark applications [54] to closely mimic existing CVEs related to control flow hijacking and ROP in MCUs [41–47] to the best of our ability. For consistency across applications, we insert the vulnerability into the last function called by the main. The vulnerability is used to overwrite $addr_{target}$ to return/jump to

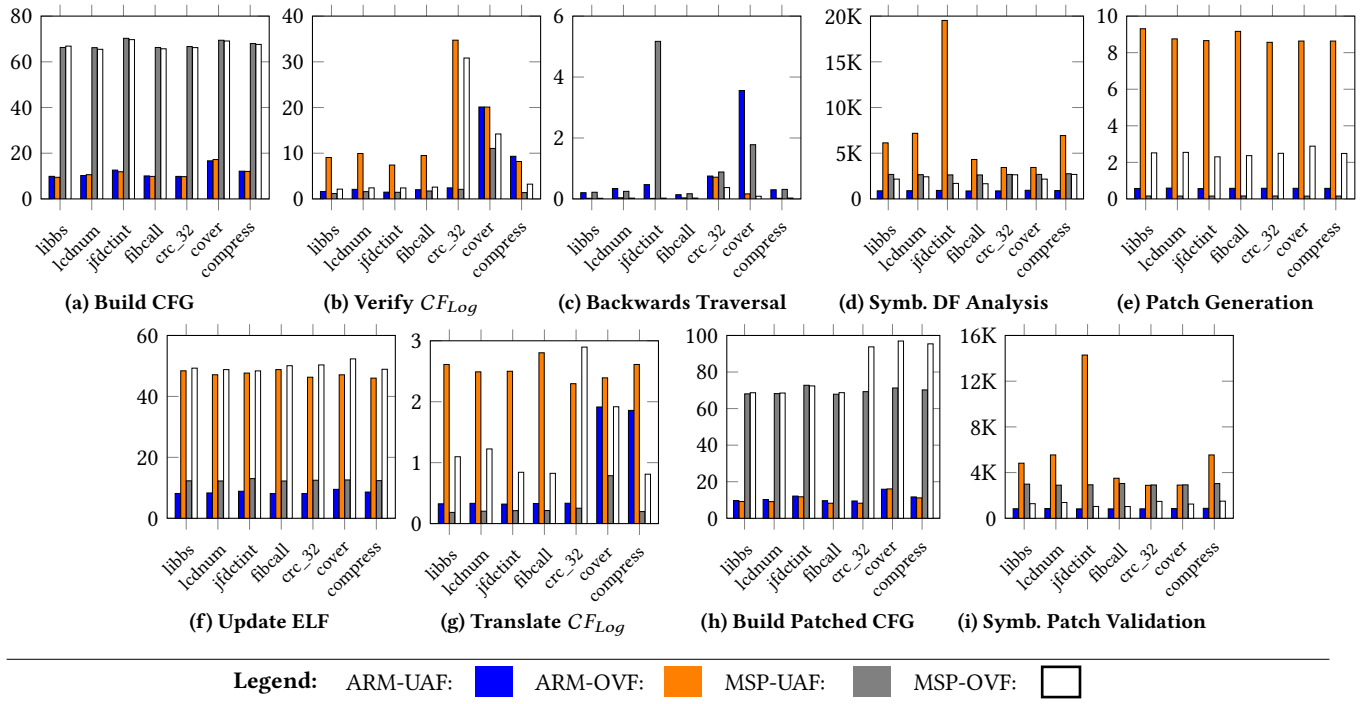


Figure 10: Run-time of *SABRE* submodules (ms) to analyze evidence of buffer overflow (OVF) and use-after-free (UAF) vulnerabilities on two MCUs and *CFA* architectures: hardware-based *CFA* atop MSP430 [12] and TEE-based *CFA* atop ARM Cortex-M33 [10].

the main function, causing an infinite loop. We execute the vulnerable programs on two unmodified *CFA* architectures – a TEE-based approach built atop ARM Cortex-M33 (TRACES [10]) and a custom hardware-based approach built atop TI MSP430 (ACFA [12]) – to collect CF_{Log}-s of the attack, which are used by *SABRE* with the modified application binary (*App.elf*) for further analysis. Fig. 10 shows the run-time of each *SABRE* sub-module during the analysis.

Static Analysis. The time required to build the CFG depends on the number of branch instructions. Building the CFG for MSP430 binaries requires more run-time, albeit smaller executables, as shown in Fig. 10a. This is due to jump destinations in disassembled MSP430 binaries being decoded with offsets, not full addresses [27]. Therefore, MSP430 binaries require additional processing steps to determine addresses from the offsets, unlike ARM binaries, which disassemble into full addresses [26]. Generating the CFG from MSP430 binaries required ≈66.27 to 70.21ms, whereas ARM binaries required ≈9.46 to 17.23ms, depending on the application.

Path Verifier. Fig. 10b presents the time to verify CF_{Log}. The verification time depends on the number of branch instructions executed by the program before the corrupted $addr_{target}$ is used. In most cases, the crafted OVF generates a larger CF_{Log} since it introduces more control flow transfers before corrupting $addr_{target}$ compared to the crafted UAF examples. Additionally, CF_{Log}-s of ARM executables (generated by TRACES [10]) require longer run-time to verify than CF_{Log}-s of MSP430 executables (generated by ACFA [12]). This is because CF_{Log}-s from TRACES do not contain any static branch instruction destinations, whereas CF_{Log}-s from ACFA contain all control flow transfers. Consequently, TRACES CF_{Log}-s require more time to verify since more processing is required by $\mathcal{V}$rf to account for the missing entries, whereas ACFA CF_{Log}-s can be naively followed by $\mathcal{V}$rf without additional effort. The detection time through CF_{Log} verification for the evaluated examples ranged from ≈1.14 to 34.74ms.

Exploit Locator. Run-time for Backwards Traversal and Symbolic DF Analysis and are shown in Fig. 10c and Fig. 10d, respectively. The Symbolic DF Analysis incurs the highest run-time from 0.868s to 19.5s. For the OVF example in ARM, CF_{Slice} of the *jfdctint* program covers the highest number of instructions (7750 in total), thus having the longest run-time. In MSP430, the *aha-compress* program (abbreviated *compress*) has the same characteristic.

Since tracking of memory and registers is not required for Backwards Traversal, its run-time is more efficient in comparison (≈0.026 to 5.31ms). For a similar reason as described above, Backwards Traversal of UAF in *jfdctint* requires the longest run-time. The *cover* application is designed to test the performance while traversing every path in multiple large switch statements. Thus, the run-time slowdown arises due to tracking definitions across the memory stores/loads of each switch case. This effect is especially apparent in ARM binaries, which implement more memory operations for switch statements than MSP430. Overall, the run-time of the Exploit Locator is mainly dominated by the Symbolic DF Analysis.

Patch Generator. Overall, the UAF patch is produced faster than the OVF patch due to its simplicity, as shown in Fig. 10e and Fig. 10f. Generating ARM Cortex-M33 patches requires more run-time due to its larger instruction set, differences in the compilation process (i.e., functions in ARM Cortex-M tend to include more loads/stores), and differences in the compiler tools.

Patch Validator. The time to translate the CF_{Log}, shown in Fig. 10g, depends on the number of new branch instructions introduced as a part of the patch. Therefore, translating CF_{Log} for the

UAF test cases requires negligible run-time (e.g., <1.0ms) since it introduces no new branch instructions, whereas the OVF test case incurs time due to new branches introduces with patch.

The time to generate the patched CFG depends on the architecture. For the UAF case, the run-time is approximately the same since the patch effectively removed one CFG node. The OVF case requires more time to rewrite additional nodes. This difference is more pronounced for MSP430 due to decoding of jump instructions, as previously described for Static Analysis.

Like the Exploit Locator, the run-time of the Patch Validator is dominated by the Symbolic DF Analysis (from 0.82s to 14.3s in Fig. 10i). Nonetheless, this analysis is performed faster in the Patch Validator because the patch causes the previously vulnerable node to exit more quickly. The bottleneck of the Symbolic DF Analysis comes from emulating memory access using *MemMap*. *SABRE* also requires an additional check for each memory write to determine whether the write corrupts $addr_{target}$ (recall Sec. 4.2). Future work could improve *SABRE*'s modeling of memory accesses.

5.2 Patch Sizes

We compare program memory size before and after the patch is installed. As a baseline, the MSP430 binaries consumed from 792 to 5764 bytes (9.90% to 72.1% of the program memory) and the ARM Cortex-M33 binaries consumed from 5.3 KB to 10.3 KB (0.31% to 2.25% of the program memory). The difference in baseline is due to the MSP430 prototype being much more resource-constrained than the ARM Cortex-M33 prototype (see Sec. 5).

After the OVF patch, the MSP430 binaries consumed 1056 to 6028 bytes (13.2% to 75.4% of the program memory) and ARM Cortex-M33 binaries consumed 5.4 KB to 10.4 KB (from 0.41% to 2.35% of the program memory). This represents an average increase of ≈264.5 bytes (≈3.31%) and ≈189.7 bytes (≈0.10%), respectively.

Since the UAF patch simply replaces a `call` instruction in place with an equivalent number of `nop`-s, it incurs no increase in the binary size. However, it incurs additional data memory overhead at run-time due to the missing `free`. The data memory overhead depends on the size of the object that is no longer freed.

6 Related Work

Control Flow Attestation. Current *CFA* approaches generate execution path evidence combining binary instrumentation with TEE support [3, 10, 67, 71, 73, 79, 88] or using custom hardware to detect/log control flow transfers [12, 20, 21, 82]. Most *CFA* literature focuses on securely generating correct evidence on a compromised $\mathcal{P}$rv, mentioning that this evidence can be used with CFG+SS [12, 53, 67, 71, 73, 88], hash-sets [1, 20, 79, 82], or execution emulation [21, 52] to perform verification. As discussed in this paper, techniques that create hash-based/hybrid evidence (**E1** and **E3** from Sec. 3) cannot always infer the precise malicious execution paths on $\mathcal{P}$rv and are not suitable for root cause analysis.

To our knowledge, only two prior efforts [14, 18] propose approaches for *CFA* verification. ZEKRA [18] proposes a *CFA* verification in zero-knowledge proofs to ensure that an untrusted $\mathcal{V}$rf does not learn secrets from CF_{Log}. RAGE [14] presents a method for path verification by training a Graph Neural Network (GNN) on CF_{Log}-s, removing the need for a CFG. Importantly, neither ZEKRA nor RAGE supports root cause analysis from received evidence, which is *SABRE*'s main goal. Although *SABRE* requires constructing a partial CFG (unlike RAGE), we show this is feasible for MCU applications; see Fig. 10a. Ammar et al. [2] provide a systematization of run-time defenses and discuss *CFA* verification as an open challenge. Unlike their discussion on the challenges in verifying *CFA* evidence, our work analyzes *CFA* evidence formats and their suitability for advanced analysis in depth. We also show how $\mathcal{V}$rf can use **E2** evidence to pinpoint and remediate root causes concretely.

Root Cause Analysis. Root Cause Analysis frameworks [37, 55, 80, 81, 83, 89] automate the process for identifying sources of software bugs or unexpected behavior. ARCUS [80] collects an Intel PT trace and an initial memory snapshot, which are used by an ARCUS server after a run-time violation to identify root cause vulnerabilities during execution emulation and to generate a report. Due to the cost of storing and transmitting large memory snapshots, ARCUS and similar proposals [37, 81, 83, 89] are unlikely to integrate feasibly with *CFA* on low-end MCUs. BENZENE [55] starts with the application binary and crash-inducing inputs, and it constructs a data-flow graph (DFG) to backtrack from crashes to track state conditions that lead to crashes and non-crashes. It then analyzes these conditions to generate likely root cause data-flow events. Unlike ACRUS and BENZENE, *SABRE* performs root cause analysis without large memory snapshots, crash inputs, or DFGs. Instead, *SABRE* combines CFG traversal with symbolic data-flow analysis to identify exploited instructions, generate a binary patch for the identified root cause, and verify its effectiveness.

Binary Rewriting and Patching. Automated binary rewriting [76] can be used for pre-deployment security measures [34, 74, 75, 85, 86] and for post-deployment patch generation [22, 23, 25, 57, 77, 78, 87]. Like prior work in this space [22, 25, 78, 87], *SABRE* patch instructions redirect execution to a secure implementation. However, *SABRE* identifies the target through root-cause analysis instead of function matching or other binary analysis methods. Real-world deployments assume the target device has a secure software update module to install the rewritten binary. For *SABRE*, no additional assumptions are required since this could be implemented as the remediation function within the run-time auditing workflow [10, 12]. Deployment challenges, like vendor-specific limits placed on software updates or flash endurance levels, would be accounted for in implementing the remediation function, and thus they are outside the scope of prior works in binary rewriting and *SABRE*.

7 Conclusion

We study how *CFA* evidence can be used to identify corrupted branch targets, pinpoint exploited memory instructions, and generate/verify binary patches. We analyze and classify types of *CFA* evidence and conclude that verbatim evidence provides $\mathcal{V}$rf with the most flexibility for remote analysis. We introduce *SABRE* to verify path validity and to locate/patch vulnerabilities using only the application binary and *CFA* evidence. We implement and evaluate *SABRE*'s public prototype [11], which demonstrates its effectiveness atop hardware-based and TEE-based *CFA* architectures.

Acknowledgments. We thank the anonymous reviewers and shepherd for their guidance.

References

[1] Tigist Abera et al. 2016. C-FLAT: control-flow attestation for embedded systems software. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 743–754.

[2] Mahmoud Ammar et al. 2024. SoK: Integrity, Attestation, and Auditing of Program Execution. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 77–77.

[3] Md Armanuzzaman et al. 2025. ENOLA: Efficient Control-Flow Attestation for Embedded Systems. *arXiv preprint arXiv:2501.11207* (2025).

[4] Roberto Baldoni et al. 2018. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)* 51, 3 (2018), 1–39.

[5] Tyler Bletsch et al. 2011. Jump-oriented programming: a new class of code-reuse attack. In *Proceedings of the 6th ACM symposium on information, computer and communications security (CCS)*. 30–40.

[6] Ferdinand Brasser et al. 2015. TyTAN: Tiny trust anchor for tiny devices. In *Proceedings of the 52nd annual design automation conference (DAC)*. 1–6.

[7] Nathan Burow et al. 2019. SoK: Shining light on shadow stacks. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 985–999.

[8] Adam Caulfield et al. 2022. ASAP: reconciling asynchronous real-time operations and proofs of execution in simple embedded systems. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*. 721–726.

[9] Adam Caulfield et al. 2024. SpecCFA: Enhancing Control Flow Attestation/Auditing via Application-Aware Sub-Path Speculation. (2024), 563–578.

[10] Adam Caulfield et al. 2024. TRACES: TEE-based Runtime Auditing for Commodity Embedded Systems. (2024), 257–270.

[11] Adam Caulfield et al. 2025. Github Repository for SABRE Prototype. https://github.com/SPINS-RG/SABRE.

[12] Adam Caulfield and otehrs. 2023. ACFA: Secure Runtime Auditing & Guaranteed Device Healing via Active Control Flow Attestation. In *32nd USENIX Security Symposium*. USENIX Association, 5827–5844.

[13] Long Cheng et al. 2019. Exploitation techniques and defenses for data-oriented attacks. In *2019 IEEE Cybersecurity Development (SecDev)*. IEEE, 114–128.

[14] Marco Chilese et al. 2024. One for All and All for One: GNN-based Control-Flow Attestation for Embedded Devices. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 203–203.

[15] Bob Lord (CISA). 2023. The Urgent Need for Memory Safety in Software Products. https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products.

[16] George Coker et al. 2011. Principles of remote attestation. *International Journal of Information Security* 10 (2011), 63–81.

[17] Crispin Cowan et al. 2000. Buffer overflows: Attacks and defenses for the vulnerability of the decade. In *Proceedings DARPA Information Survivability Conference and Exposition (DISCEX)*, Vol. 2. IEEE, 119–129.

[18] Heini Bergsson Debes et al. 2023. ZEKRA: Zero-Knowledge Control-Flow Attestation. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*. 357–371.

[19] Ioannis Deligiannis and George Kornaros. 2016. Adaptive memory management scheme for MMU-less embedded systems. In *11th Symposium on Industrial Embedded Systems (SIES)*. IEEE, 1–8.

[20] Ghada Dessouky et al. 2017. LO-FAT: Low-overhead control flow attestation in hardware. In *Proceedings of the 54th Annual Design Automation Conference (DAC)*. 1–6.

[21] Ghada Dessouky et al. 2018. Litehax: lightweight hardware-assisted attestation of program execution. In *International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.

[22] Ruian Duan et al. 2019. Automating Patching of Vulnerable Open-Source Software Versions in Application Binaries.. In *Network and Distributed System Security (NDSS) Symposium*.

[23] Gregory J Duck et al. 2020. Binary rewriting without control flow recovery. In *Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation (PLDI)*. 151–163.

[24] Karim Eldefrawy et al. 2012. SMART: Secure and Minimal Architecture for (Establishing Dynamic) Root of Trust. In *Network and Distributed System Security (NDSS) Symposium*, Vol. 12. 1–15.

[25] Karim Eldefrawy, Michael Locasto, Norrathep Rattanavipanon, and Hassen Saidi. 2020. Towards Automated Augmentation and Instrumentation of Legacy Cryptographic Executables. In *Applied Cryptography and Network Security (ACNS)*. Springer, 364–384.

[26] Free Software Foundation. 2024. arm-none-eabi-objdump man page. https://manpages.debian.org/unstable/binutils-arm-none-eabi/arm-none-eabi-objdump.1.en.html

[27] Free Software Foundation. 2024. msp430-objdump man page. https://manpages.debian.org/testing/binutils-msp430/msp430-objdump.1.en.html

[28] Python Software Foundation. 2024. Python libary pyelftools. https://pypi.org/project/pyelftools/0.20/

[29] Munir Geden and Kasper Rasmussen. 2019. Hardware-assisted remote runtime attestation for critical embedded systems. In *17th International Conference on Privacy, Security and Trust (PST)*. IEEE, 1–10.

[30] Intel. 2015. Intel Processor Trace. https://edc.intel.com/content/www/us/en/design/ipla/software-development-platforms/client/platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/010/intel-processor-trace/. [Online; accessed 14-March-2025].

[31] Hakan Kayan et al. 2022. Cybersecurity of industrial cyber-physical systems: a review. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–35.

[32] Rick Kennell and Leah H Jamieson. 2003. Establishing the genuinity of remote computer systems. In *12th USENIX Security Symposium*.

[33] Keystone. 2024. Keystone: the ultimate assembler. https://www.keystone-engine.org/

[34] Pantea Kiaei et al. 2021. Rewrite to reinforce: Rewriting the binary to apply countermeasures against fault injection. In *58th Design Automation Conference (DAC)*. IEEE, 319–324.

[35] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.

[36] Xeno Kovah et al. 2012. New results for timing-based attestation. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 239–253.

[37] Runhao Liu et al. 2022. SEEKER: A root cause analysis method based on deterministic replay for multi-type network protocol vulnerabilities. In *International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 131–138.

[38] Miguel Masmano et al. 2003. Dynamic storage allocation for real-time embedded systems. *Proc. of Real-Time System Simposium WIP* (2003).

[39] The MITRE Corporation (MITRE). 2024. 2024 CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html.

[40] Muhammad Nouman Nafees et al. 2023. Smart grid cyber-physical situational awareness of complex operational technology attacks: A review. *Comput. Surveys* 55, 10 (2023), 1–36.

[41] National Vulnerability Database. 2017. CVE-2017-14201. https://nvd.nist.gov/vuln/detail/CVE-2017-14201 Accessed: 2025-May-04.

[42] National Vulnerability Database. 2020. CVE-2019-16127. https://nvd.nist.gov/vuln/detail/CVE-2019-16127 Accessed: 2025-May-04.

[43] National Vulnerability Database. 2020. CVE-2020-10019. https://nvd.nist.gov/vuln/detail/CVE-2020-10019 Accessed: 2025-May-04.

[44] National Vulnerability Database. 2020. CVE-2020-10023. https://nvd.nist.gov/vuln/detail/CVE-2020-10023 Accessed: 2025-May-04.

[45] National Vulnerability Database. 2021. CVE-2021-0920. https://nvd.nist.gov/vuln/detail/CVE-2021-0920 Accessed: 2025-May-04.

[46] National Vulnerability Database. 2021. CVE-2021-35395. https://nvd.nist.gov/vuln/detail/CVE-2021-35395 Accessed: 2025-May-04.

[47] National Vulnerability Database. 2022. CVE-2022-34835. https://nvd.nist.gov/vuln/detail/CVE-2022-34835 Accessed: 2025-May-04.

[48] Antonio Joia Neto and Ivan De Oliveira Nunes. 2023. ISC-FLAT: On the Conflict Between Control Flow Attestation and Real-Time Operations. In *29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 133–146.

[49] Job Noorman et al. 2017. Sancus 2.0: A low-cost security architecture for iot devices. *ACM Transactions on Privacy and Security (TOPS)* 20, 3 (2017), 1–33.

[50] Ivan De Oliveira Nunes et al. 2019. VRASED: A Verified Hardware/Software Co-Design for Remote Attestation. In *28th USENIX Security Symposium*. 1429–1446.

[51] Ivan De Oliveira Nunes et al. 2020. APEX: A verified architecture for proofs of execution on remote devices under full software compromise. In *29th USENIX Security Symposium*. 771–788.

[52] Ivan De Oliveira Nunes et al. 2021. Dialed: Data integrity attestation for low-end embedded devices. In *58th Design Automation Conference (DAC)*. IEEE, 313–318.

[53] Ivan De Oliveira Nunes et al. 2021. Tiny-CFA: Minimalistic control-flow attestation using verified proofs of execution. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 641–646.

[54] James Pallister et al. 2013. BEEBS: Open benchmarks for energy measurements on embedded platforms. *arXiv preprint arXiv:1308.5174* (2013).

[55] Younggi Park et al. 2024. BENZENE: A Practical Root Cause Analysis System with an Under-Constrained State Mutation. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 1865–1883.

[56] Nick L Petroni Jr et al. 2004. Copilot-a coprocessor-based kernel runtime integrity monitor.. In *USENIX security symposium*. San Diego, USA, 179–194.

[57] Prashant Hari Narayan Rajput et al. 2023. ICSPatch: Automated Vulnerability Localization and {Non-Intrusive} Hotpatching in Industrial Control Systems using Data Dependence Graphs. In *32nd USENIX Security Symposium*. 6861–6876.

[58] M Ramakrishna et al. 2008. Smart dynamic memory allocator for embedded systems. In *23rd International Symposium on Computer and Information Sciences (ISCIS)*. IEEE, 1–6.

[59] Ganesan Ramalingam. 1994. The undecidability of aliasing. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 16, 5 (1994), 1467–1471.

[60] Realtek Semiconductor Corp. 2021. Realtek AP-Router SDK Advisory. https://www.realtek.com/images/safe-report/Realtek_APRouter_SDK_Advisory-CVE-2021-35392_35395.pdf Accessed: 2025-May-04.

[61] Ryan Roemer et al. 2012. Return-oriented programming: Systems, languages, and applications. *ACM Transactions on Information and System Security (TISSEC)* 15, 1 (2012), 1–34.
[62] Dries Schellekens, Brecht Wyseur, and Bart Preneel. 2008. Remote attestation on legacy operating systems with trusted platform modules. *Science of Computer Programming* 74, 1-2 (2008), 13–22.
[63] Arvind Seshadri et al. 2004. SWATT: Software-based attestation for embedded devices. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 272–282.
[64] Arvind Seshadri et al. 2005. Pioneer: verifying code integrity and enforcing untampered code execution on legacy systems. In *Proceedings of the twentieth ACM symposium on Operating Systems Principles (SOSP)*. 1–16.
[65] Arvind Seshadri et al. 2008. SAKE: Software attestation for key establishment in sensor networks. In *Distributed Computing in Sensor Systems (DCOSS)*. 372–385.
[66] Hovav Shacham et al. 2004. On the effectiveness of address-space randomization. In *Proceedings of the 11th ACM conference on Computer and communications security (CCS)*. 298–307.
[67] Zhichuang Sun et al. 2020. OAT: Attesting operation integrity of embedded devices. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 1433–1449.
[68] Swiftloke. 2024. Github Repository for MSProbe. https://github.com/Swiftloke/MSProbe
[69] Xi Tan et al. 2024. SoK:Where's the "up"?! A Comprehensive (bottom-up) Study on the Security of Arm Cortex-M Systems. In *18th USENIX WOOT Conference on Offensive Technologies (WOOT)*. 149–169.
[70] SymPy Development Team. 2024. sympy. https://docs.sympy.org/latest/index.html
[71] Flavio Toffalini et al. 2019. ScaRR: Scalable Runtime Remote Attestation for Complex Systems. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 121–134.
[72] Jo Vliegen et al. 2019. SACHa: Self-attestation of configurable hardware. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 746–751.
[73] Jinwen Wang et al. 2023. ARI: Attestation of Real-time Mission Execution Integrity. In *32nd USENIX Security Symposium*. 2761–2778.
[74] Minghua Wang et al. 2015. Binary code continent: Finer-grained control flow integrity for stripped binaries. In *Proceedings of the 31st annual computer security applications conference (ACSAC)*. 331–340.
[75] Richard Wartell et al. 2012. Securing untrusted code via compiler-agnostic binary rewriting. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 299–308.
[76] Matthias Wenzl et al. 2019. From hack to elaborate technique—a survey on binary rewriting. *ACM Computing Surveys (CSUR)* 52, 3 (2019), 1–37.
[77] David Williams-King et al. 2020. Egalito: Layout-agnostic binary recompilation. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 133–147.
[78] Zhengzi Xu et al. 2020. Automatic hot patch generation for android kernels. In *29th USENIX Security Symposium*. 2397–2414.
[79] Nikita Yadav and Vinod Ganapathy. 2023. Whole-Program Control-Flow Path Attestation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2680–2694.
[80] Carter Yagemann et al. 2021. ARCUS: symbolic root cause analysis of exploits in production systems. In *30th USENIX Security Symposium*. 1989–2006.
[81] Carter Yagemann et al. 2021. Automated bug hunting with data-driven symbolic root cause analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 320–336.
[82] Shaza Zeitouni et al. 2017. ATRIUM: Runtime attestation resilient under memory attacks. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 384–391.
[83] Jun Zeng et al. 2022. Palantír: Optimizing attack provenance with hardware-enhanced system observability. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 3135–3149.
[84] Zephyr Project. 2016. Zephyr Repository. https://github.com/zephyrproject-rtos. Accessed: 2025-May-04.
[85] Chao Zhang et al. 2013. Practical control flow integrity and randomization for binary executables. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 559–573.
[86] Mingwei Zhang and R Sekar. 2015. Control flow and code integrity for COTS binaries: An effective defense against real-world ROP attacks. In *Proceedings of the 31st Annual Computer Security Applications Conference (ACSAC)*. 91–100.
[87] Xuewen Zhang et al. 2017. Embroidery: Patching vulnerable binary code of fragmentized android devices. In *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 47–57.
[88] Yumei Zhang et al. 2021. ReCFA: resilient control-flow attestation. In *Annual Computer Security Applications Conference (ACSAC)*. 311–322.
[89] Hao Zhou et al. 2022. NCScope: hardware-assisted analyzer for native code in android apps. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 629–641.

APPENDIX

A Alternative Prover Configurations

Alternative mechanisms such as Address Space Layout Randomization (ASLR) [66] or *CFA* optimizations (e.g., SpecCFA [9]) may be concurrently employed on $\mathcal{P}$rv for increased security/performance. In both cases, *SABRE* remains effective as long as $\mathcal{V}$rf knows the configuration of the respective mechanism. For example, for *CFA* in MCUs (with no MMU), ASLR would be performed before $\mathcal{P}$rv deployment. Therefore, $\mathcal{V}$rf would have access to the randomized binary. Since an ASLR bypass would corrupt a pointer to a memory address post-randomization, *SABRE* would not need to perform any additional handling and can pass the randomized binary with the corresponding **E2** evidence to *SABRE*. *CFA* optimizations on **E2** evidence, including speculation-based ones such as SpecCFA [9], are lossless by design. In it, $\mathcal{V}$rf replaces expected subpaths with reduced-sized reserved symbols. Since the subpath-to-symbol correspondence is known by $\mathcal{V}$rf, its ability to analyze received evidence is not lost, regardless of this optimization.

B Limitations and Future Work

Scaling to Complex Systems. Most current *CFA* and run-time auditing targets MCUs [2]. Conversely, *SABRE* considers MCU binaries (i.e., MSP430 and ARM Cortex-M33). An open problem in this space is scaling $\mathcal{P}$rv RoTs from low-end MCUs to complex systems. Assuming *CFA* and generation of **E2** in complex systems is possible, a scalability challenge would come from performing *SABRE*'s symbolic data-flow analysis. Although *SABRE* does not face typical path explosion risk in symbolic execution due to its use of concrete run-time traces to guide the execution, a unique concern would arise when emulating the active state (memory and registers) during symbolic data flow analysis, which for MCU binaries is already the bottleneck in *SABRE*'s Exploit Locator (recall Sec. 5.1). Optimizing *SABRE*'s state modeling during symbolic data-flow analysis is an interesting avenue for future work.

Adding Vulnerabilities and Attacks. This work focuses on control flow hijacking and code-reuse attacks. For this reason, we tailored our framework towards the most prevalent root-cause memory vulnerabilities for these attacks: use-after-free vulnerabilities and buffer overflow-induced out-of-bounds writes [39]. Additionally, these memory vulnerabilities are prevalent for launching arbitrary code execution in real-world MCU software (as discussed in Sec. 2.2). However, other memory vulnerabilities (e.g., signedness errors, integer overflows, and double frees) could lead to arbitrary code execution. In these cases, *SABRE* would determine that a generated patch was ineffective and generate a report to provide a starting point for manual inspection (as discussed in Sec. 4.4). Similarly, *SABRE* does not support data-oriented attacks (e.g., direct-data-manipulation or data-oriented-programming [13]) and would fail in the same manner in its attempts to generate a patch or pass **E2** verification if the attack did not corrupt any control-data. These attacks could be detected through *DFA* run-time evidence that includes data inputs (recall Sec. 2.3). Future work could extend *SABRE* to detect/patch other memory vulnerabilities and to use data inputs from *DFA* to address data-oriented attacks.